University of Bologna

Implementing PTB CLI via TypeScript

Interface for Programmable Transaction Blocks on IOTA Blockchain

Author: Giovanni Maria Savoca

Professor: Prof. Stefano Ferretti

Tutor: Arianna Arruzzoli

Academic Year: 2025/2026

February 2026

Contents

1 Introduction **2**

2 Project Objectives **2**

3 System Architecture **2**

3.1 Technologies Used . . . 3
3.2 IOTA SDK Components . . . 3
3.2.1 IotaClient . . . 3
3.2.2 Transaction (PTB Builder) . . . 3
3.2.3 Ed25519Keypair . . . 4
3.3 User Interface Structure . . . 4
3.3.1 Header and Wallet Information . . . 5
3.3.2 Left Column: Tabs and Function Cards . . . 5
3.3.3 Right Column: Results and Transaction Details . . . 5
3.3.4 Status Banner . . . 6
3.4 Available Move Functions . . . 6
3.4.1 Module `myutils` — String Utilities . . . 6
3.4.2 Module `date_time` . . . 7
3.4.3 Module `manage_documents` . . . 7
3.5 Transaction Construction and Execution . . . 7
3.5.1 Simulation Mode (DevInspect) . . . 7
3.5.2 Real Execution Mode (ExecuteTransaction) . . . 8
3.6 Modular Architecture and Refactoring . . . 8
3.6.1 Logical Flow of Transactions . . . 9
3.7 Justification of Implementation Choices . . . 9

4 Results **9**

5 Conclusions **10**

1 Introduction

Over the past decade, the adoption of blockchain technology has grown significantly, leading to a wide range of use cases. Its intrinsic properties have been studied to enable the development of numerous applications globally. From finance to gaming, to healthcare and public administration, blockchain has become increasingly understood and applied in various contexts.

Applications leveraging decentralized technologies are commonly called **decentralized applications** (dApps). These employ blockchain for data storage and processing, ensuring decentralization while functioning similarly to traditional applications. Recently, there has been a trend toward improving decentralization by using modular components for specialized functions. To allow interaction with the blockchain, IOTA have implemented **Programmable Transaction Blocks** (PTB), a tool that uses the Move language to interact with smart contracts.

The present project aims to implement an SDK in TypeScript to work with smart contracts already deployed on the IOTA Testnet. The idea is to develop a basic interface (or a form) to manage and transmit data to the on-chain logic, providing a clean and simple interface for the user.

2 Project Objectives

The main objective is to work with PTB and TypeScript to create a module that allows interaction between the user and the blockchain. The project is structured into three specific milestones that served as evaluation criteria:

- **Milestone 1:** Create a basic interface to interact with the `utils` package. The interface includes a form for data entry, buttons for different functions, and a colored banner to display the response status (success, error, or info).
- **Milestone 2:** Implementation of Move simple and complex object types in the form. This includes recursive management of numbers (from `u8` to `u256`), strings, and object references.
- **Milestone 3:** Expansion of the input format to include mutable and immutable objects, vectors, and `VecMap` structures.
- **Milestone 4 (Bonus Implementation):** Advanced integration with the `manage_documents` package, supporting collaborative document creation with multi-author permission management.

The project satisfied these requirements by providing a modular and extensible TypeScript SDK integrated into a user-friendly web interface.

3 System Architecture

This section provides a comprehensive description of the system's architecture, explaining the technology stack, the role of each component, the structure of the user interface, the complete catalogue of available functions, and the mechanisms through which transactions are built, executed, and displayed.

3.1 Technologies Used

The project utilizes the following technologies and tools required:

- **IOTA Blockchain (Testnet):** The foundational layer for executing Move smart contracts.
- **Move Language:** The resource-oriented language used for the logic of the deployed packages.
- **TypeScript:** Used to build the SDK and the user interface, providing type safety for blockchain interactions.
- **@iota/iota-sdk:** The official integration layer to build PTBs and communicate with the network.
- **IOTA Explorer (Testnet Scan):** Used to monitor transactions and verify the **digest**, which describes the network's interpretation of inputs and outputs.
- **Vite:** For development environment management and production bundling.

3.2 IOTA SDK Components

The IOTA SDK provides three fundamental components that constitute the integration layer between the web application and the blockchain:

3.2.1 IotaClient

The `IotaClient` class manages the JSON-RPC connection to an IOTA full node. It is instantiated once at application startup by calling the helper function `getFullnodeUrl("testnet")`, which returns the public Testnet RPC endpoint.
All subsequent interactions with the blockchain — simulations, real executions, and object queries — pass through this single client instance. The client exposes two key methods used by the application: `devInspectTransactionBlock` for read-only simulations and `executeTransactionBlock` for state-modifying operations.

3.2.2 Transaction (PTB Builder)

The `Transaction` class is the core builder for Programmable Transaction Blocks (PTBs). A PTB is not a single operation but a sequence of composable *commands* that are executed atomically: either all commands succeed, or the entire block is rolled back. This is one of the most distinctive features of IOTA's transaction model, since it allows chaining multiple operations — such as creating an object, modifying it, and transferring it — within a single on-chain transaction.

Each user action in the interface creates a new `Transaction` instance and populates it with commands. The primary command used in this project is `moveCall`, which invokes a specific function in a deployed Move module. The target is specified as a string in the format `package::module::function` (e.g., `0x578...::myutils::split`), and the arguments are provided as typed references.

The SDK distinguishes two categories of transaction inputs:

- **Pure inputs** (`tx.pure.*`): Represent literal values that do not exist on-chain. These include primitives like `tx.pure.u8()`, `tx.pure.u64()`, `tx.pure.string()`, `tx.pure.bool()`, composite types like `tx.pure.vector("address", [...])` for typed vectors, and `tx.pure.address()` for blockchain addresses. Each method serializes the JavaScript value into the corresponding BCS encoding expected by the Move runtime.
- **Object inputs** (`tx.object()`): Reference on-chain objects by their Object ID. The SDK resolves the object's ownership mode (Immutable, Owned, or Shared) automatically at build time. A notable example is the Clock object (`0x6`), a system-level Shared object used to provide timestamps.

Once all commands are configured, the `build()` method serializes the entire transaction into BCS bytes. If the flag `onlyTransactionKind: true` is passed, only the command payload is serialized (suitable for simulation via `devInspect`); otherwise, a complete transaction is produced, ready for signing and submission.

3.2.3 Ed25519Keypair

The `Ed25519Keypair` class manages the cryptographic identity of the user. Ed25519 is an elliptic-curve digital signature algorithm based on the Edwards curve Curve25519, chosen by IOTA for its combination of security, speed, and compact signature size (64 bytes).

The application loads a 32-byte private key from the `.env` file, where it is stored as a hexadecimal string under the variable `VITE_PRIVATE_KEY_HEX`. At startup, the hex string is decoded into a `Uint8Array` of raw bytes and passed to `Ed25519Keypair.fromSecretKey()`, which derives the corresponding public key and creates the keypair object.

The keypair fulfills two roles in the application:

- **Address derivation:** The method `getPublicKey().toIotaAddress()` derives the IOTA address from the public key. This address uniquely identifies the wallet on the network and is used as the transaction sender, displayed in the interface header, and pre-filled in the document author field.
- **Transaction signing:** When a real execution is requested, the method `signTransaction()` takes the BCS-serialized transaction bytes and produces a cryptographic signature. This signature is sent alongside the transaction bytes to the IOTA node, which verifies it against the sender's public key before accepting the transaction.

For simulated transactions (`devInspect`), no signature is required — the node only needs a sender address to resolve object ownership, but does not verify authenticity. This distinction is what allows cost-free testing without exposing the private key to unnecessary signing operations.

3.3 User Interface Structure

The web interface adopts a **two-column layout** with a tab-based navigation system. The left column hosts user inputs and function controls, while the right column — which remains fixed during scrolling — displays results and transaction details.

3.3.1 Header and Wallet Information

At the top of the page, a header displays the application title and a brief description. Immediately below, a bar shows the currently loaded wallet address (truncated for readability) alongside a `Request Faucet` button. Clicking this button invokes the IOTA Testnet faucet API to credit test tokens to the wallet, enabling real transaction execution.

3.3.2 Left Column: Tabs and Function Cards

The left column is organized into two tabs, selectable via navigation buttons:

Tab "Utils Functions". This tab presents a scrollable list of *function cards*, one for each Move function available in the `utils` package. Each card contains:

- The function name and its Move signature (e.g., `split(str, separator)` → `vector<String>`).
- One or more labeled input fields, pre-configured with appropriate types (text fields for strings, number fields for integers, comma-separated fields for vectors).
- A `SIMULATE` button to test the function via `devInspect` (no gas cost, read-only).
- An `EXECUTE` button, initially disabled, that becomes active only after a successful simulation. Clicking it submits the transaction to the blockchain with a real signature and gas payment.

Tab "Create Document". This tab presents a single complex form for creating an on-chain document. The form includes fields for title, author address (with a `Fill` button to auto-populate the wallet address), content (a multi-line text area), category, and version. A dynamic section labeled `Additional Editors` allows the user to add an arbitrary number of collaborator addresses through the `+ Add` button; each added row can be individually removed. The form provides both a `SIMULATE` button and an `EXECUTE` button.

A `Clear All` button at the bottom of the left column resets every input field, clears results, and hides the status banner.

3.3.3 Right Column: Results and Transaction Details

The right column contains two panels that remain visible (sticky positioning) regardless of scrolling in the left column:

Results panel. Displays the name of the last invoked function and its decoded return value. When a real transaction has been executed, it also shows a clickable link to the IOTA Explorer page for the corresponding Transaction Digest. A `Copy` button allows copying the result to the clipboard.

Transaction Details panel. Shows the complete JSON response returned by the IOTA node. For simulation results, this includes the full `effects` and `results` objects with raw return values. For real executions, it additionally shows the transaction inputs, object changes, emitted events, and gas consumption. A search bar at the top of this panel enables real-time text search within the JSON payload, with match highlighting and a counter.

3.3.4 Status Banner

A notification banner spans the full width of the page and provides immediate visual feedback after any operation. The banner uses color coding: **green** for successful operations, **red** for errors, and **blue** for informational messages. Each banner can be dismissed by clicking its close button.

3.4 Available Move Functions

The interface exposes eleven utility functions from two Move modules, plus one document management function. Each function is accessible through a dedicated card in the interface. The following subsections describe every function, its purpose, its input parameters, and its return type.

3.4.1 Module `myutils` — String Utilities

split. Splits a string into a vector of substrings using a single-character separator specified by its ASCII code. The function iterates over the bytes of the input string, identifies separator positions, and extracts substrings using the `substring` method. Input: a UTF-8 string and a `u8` separator code (e.g., 44 for comma). Output: `vector<String>`.

check_lowercase. Verifies whether all characters in a string are lowercase ASCII letters (codes 97–122). Input: a UTF-8 string. Output: `bool`.

check_is_hex. Checks whether a string represents a valid hexadecimal sequence. The function first verifies that the string length is even, then checks that every byte falls within the ranges 97–122 (lowercase letters a–z) or 48–57 (digits 0–9). Input: a UTF-8 string. Output: `bool`.

remove_white_spaces. Removes all whitespace characters (spaces, tabs, line feeds, carriage returns) from a string. The function uses a split-and-rejoin approach: it identifies whitespace positions, splits the string into tokens, and concatenates them without gaps. Input: a UTF-8 string (passed by reference). Output: `String`.

equals. Compares two strings for equality after removing whitespace from both. It first validates length equality, then performs character-by-character comparison using ASCII conversion. Input: two UTF-8 strings (passed by reference). Output: `bool`.

count_digits. Counts the number of decimal digits in an unsigned 64-bit integer. The function repeatedly divides the number by 10, incrementing a counter at each step. Returns 1 for the special case of zero. Input: `u64`. Output: `u64`.

sum_bytes_u8. Computes the sum of all elements in a byte vector, returning the result as a `u8`. To prevent overflow, the function uses saturating addition: if the cumulative sum would exceed 255, it is capped at 255. Input: `vector<u8>` (passed by reference). Output: `u8`.

sum_bytes_u32. Computes the sum of all elements in a byte vector, returning the result as a `u32`. Unlike the `u8` variant, this function uses assertion-based overflow protection: it aborts the transaction if the sum exceeds the maximum `u32` value. Input: `vector<u8>` (passed by reference). Output: `u32`.

string_to_u64. Attempts to convert a numeric string (e.g., "12345") into an unsigned 64-bit integer. The function processes each character, verifies that it is a valid ASCII digit, checks for overflow, and accumulates the result using the formula value = value×10+digit. Input: a UTF-8 string (passed by reference). Output: `Option<u64>` (returns `None` if the string is not a valid number or would overflow).

boolean_from_str. Converts a string representation of a boolean ("true", "True", "false", "False") into the corresponding Move boolean value. Input: a UTF-8 string (passed by reference). Output: `Option<bool>` (returns `None` if the string does not match any recognized pattern).

3.4.2 Module `date_time`

ms_to_string. Converts a Unix timestamp in milliseconds to a human-readable date string in the format `YYYY-MM-DD`. Internally, the function computes the total number of days since the Unix epoch (January 1, 1970), iteratively determines the year (accounting for leap years), the month (using per-month day counts), and the day. Months and days below 10 are zero-padded. Input: `u64` (milliseconds since epoch). Output: `String`.

3.4.3 Module `manage_documents`

create_document. Creates a persistent document object on the blockchain. This function belongs to a separate package (`manage_documents`) and is invoked with eight arguments: title, a vector of author addresses, content, category, version, the on-chain Clock object (address `0x6`), a boolean flag for public visibility, and a metadata map. The created object is a `Shared` object, meaning that all addresses listed in the author vector gain permission to interact with it in future transactions.

3.5 Transaction Construction and Execution

The application provides two distinct execution paths for sending transactions to the IOTA network. The choice between them depends on whether the operation needs to modify blockchain state.

3.5.1 Simulation Mode (DevInspect)

When the user clicks a `SIMULATE` button, the application follows this pipeline:

1. A new `Transaction` object is created.
2. The appropriate *function builder* retrieves input values from the DOM, validates them (checking for empty fields, invalid formats, or out-of-range numbers), and adds a `moveCall` instruction to the transaction.
3. The sender address is set on the transaction, and it is serialized to bytes using `onlyTransactionKind: true` (no signature is required).
4. The serialized bytes are sent to the IOTA node via `devInspectTransactionBlock`, which executes the transaction in a sandboxed environment without modifying state or consuming gas.

5. The node returns the full execution result, including `effects`, `status`, and — crucially — the `returnValues` array, which contains the raw BCS-encoded bytes of each value returned by the Move function.
6. The application decodes the return values (see below) and displays them in the Results panel.

This mode is ideal for **pure functions** (those that only compute and return values without modifying on-chain state). It allows unlimited, cost-free testing.

3.5.2 Real Execution Mode (ExecuteTransaction)

When the user clicks an `EXECUTE` button (which is only enabled after a successful simulation), the application follows a different pipeline:

1. The transaction is built as before, but this time without the `onlyTransactionKind` flag, producing a complete transaction payload.
2. The transaction bytes are signed using the `Ed25519Keypair` loaded from the `.env` file.
3. The signed transaction is sent to the node via `executeTransactionBlock`. The SDK options `showEffects`, `showEvents`, `showInput`, and `showObjectChanges` are all enabled, requesting the node to return the most comprehensive response possible.
4. The node validates the signature, checks that the sender has sufficient gas, executes the transaction, and — if successful — commits the state changes to the blockchain.
5. The application receives a response containing the Transaction Digest (a unique hash), the list of created or modified objects, emitted events, and the total gas consumed.
6. The Results panel displays the digest and a direct link to the IOTA Explorer for verification.

A confirmation dialog is shown before every real execution to warn the user that this operation will consume gas tokens.

3.6 Modular Architecture and Refactoring

A significant evolution of the project was the transition from a monolithic script to a **modular architecture**. This refactoring was necessary to manage the increasing complexity of the milestones and ensure code reusability. The codebase was split into specialized modules:

- **main.ts:** Entry point for application initialization and event routing.
- **tx-engine.ts:** Abstracts SDK details, handling both simulated and real executions.
- **ui-interface.ts:** Manages DOM selectors, banners, and result display.
- **utils-manager.ts:** Registry of function builders for the `utils` package.
- **doc-manager.ts:** Logic for document creation and multi-editor management.

3.6.1 Logical Flow of Transactions

The logical flow follows a strict **Verification-before-Execution** pattern, ensuring a clean interaction between the user and the blockchain logic:

1. **Input Capture:** Data is retrieved from the form using `ui-interface.ts`.
2. **Validation:** `utils-manager.ts` validates types (e.g., `u8` to `u256`) and address formats.
3. **PTB Construction:** A `Transaction` block is built via `moveCall`.
4. **Simulation (DevInspect):** The block is simulated on the node to verify correctness without gas costs.
5. **Execution:** After a successful simulation, the real transaction is signed and submitted, and the resulting **digest** is displayed for tracking on the IOTA Explorer.

3.7 Justification of Implementation Choices

The implementation was guided by the need to satisfy the professor's milestones with a professional approach:

- **Choosing TypeScript:** Essential for handling Move types correctly. The ability to use `BigInt` for `u256` and strong typing for PTB arguments prevents runtime errors during complex interactions.
- **Refactoring to Modular Structure:** Transitioning from a single `main.ts` to five specialized modules allowed for better management of high-complexity features like vectors and `VecMap` (Milestone 3).
- **Mandatory Simulation:** This design choice protects the user from failed transactions, ensuring the SDK logic is correct before interacting with the live Testnet.
- **Registry Pattern:** Using a central registry for function builders (`utils-manager.ts`) made it easy to expand support for the various functions required in Milestone 1 and 2.

4 Results

All three milestones defined by the professor have been successfully completed and verified:

- **Milestone 1:** Functional interface with forms, buttons, and status banners.
- **Milestone 2:** Support for all numeric types (`u8-u256`), strings, and object references.
- **Milestone 3:** Successful integration of mutable/immutable objects, vectors, and `VecMap` structures.
- **Milestone 4 (Bonus):** Deployment and verification of a multi-author document management system, proving the system's ability to handle complex state-modifying transactions.

Transactions have been verified through the IOTA Explorer, confirming the correct execution of PTBs and the storage of data on the Testnet.

5 Conclusions

The project demonstrated how PTBs and TypeScript can be combined to create powerful tools for blockchain interaction. The transition to a modular architecture (*rielaborazione*) was a key factor in handling the complexity of the requirements.

The final result is a robust SDK and interface that allows for safe, clear, and simple user interaction with Move smart contracts on the IOTA network, meeting all the academic criteria and providing a solid foundation for future dApp development.

References

IOTA Documentation https://docs.iota.org/

IOTA Explorer https://iotascan.com/testnet/home

VecMap Reference https://docs.iota.org/developer/references/framework/iota/vec_map

Move Language https://move-language.github.io/move/